# Agents need a map of the source first

Paths say too little · whole sources cost too much context · a map says where to look

## Without a map

The agent sees file names, greps,
and opens file after file.

Context fills with code the task
never needed.

Large or unfamiliar files (SQL dumps,
XML, scripts) are skipped or cut off.
Every new session starts from zero.

## With a map: zoom in

| | | |
|---|---|---|
| 1 | **project.ai.md** | one line per package |
| 2 | **package.ai.md** | one line per file |
| 3 | **<file>.ai.md** | purpose, API, exact facts |
| 4 | **the source** | only where exact detail is needed |

## srcmorph builds this map for any source tree

One Markdown summary per file, package and project · Java, SQL, shell, Perl, XML and any other text file

Local llama.cpp models or any OpenAI-compatible API · Maven plugin or CLI

Summaries are lossy: the map says where to look, the source stays the reference.

# srcmorph: a navigable map of the source tree

Project → package → file summaries · every kind of source · example: the benchmark corpus

## project.ai.md

Project map · one line per package

**java/config / package.ai.md**

- AiConditionEvaluator.java.ai.md
- AiFactExtractor.java.ai.md

**sql/library / package.ai.md**

- 01_schema.sql.ai.md
- 03_routines.sql.ai.md

**other/scripts / package.ai.md**

- rotate_backups.sh.ai.md
- sales_report.py.ai.md

**other/ringbuf / package.ai.md**

- ringbuf.c.ai.md
- ringbuf.h.ai.md

The benchmark corpus: 24 files in 12 packages, 1 project map

## Give agents a map first

Start with the project · zoom into relevant packages

Read original files when exact detail is needed

## Every entry starts with a one-line lead

The package and project maps are built from these leads, so each level reads like a legend of the level below.

## Exact facts beside the summary

Counted by the profile's regular expressions over the whole file, not by the model: tables, functions, elements, dependencies ...

## Stays current in CI

A source checksum in every header: a changed file rewrites its entry and the packages above it; unchanged entries stay as they are.

Summaries are lossy. Actual agent token savings depend on the task and selected context.

# One entry of the map

An 870 KB XML catalog, summarized by GPT-6 Luna in three chunks · large-file corpus, 2026-11-02

```
### catalog.xml
- C: CCF59A7E
- X: file
- P: 80AA1478
- N: C0493A8E
---
**Facts (exact, whole file):** elements: 20291;
  namespace declarations: 2; id and name attributes: 80
**Coverage:** 3 of 3 chunks summarized
> Defines a 40-category product catalog with EUR
  pricing, warehouse inventory, and recall details
#### Definitions
- Each category contains 125 product records;
  5,000 products total, SKU-001-0001 .. SKU-040-0125.
- Products in cat-020 and cat-021 also contain
  inv:recall: 250 recall records.
#### Configuration · #### Dependencies · ...
```

### Header

Source checksum (C), the prompt (P) and the fact counters (N) it was written with

### Facts line

Exact counts over the whole file, by the profile's regular expressions

### Coverage line

What the model read: every chunk, or a sample of a very large file

### Lead and sections

The lead goes into the package map; the sections depend on the file kind

Excerpt: five header lines (format version, dates, plugin and AI version) and most sections left out.

# Any kind of source: a prompt and exact facts per kind

Rules route each file by path, extension and size · one profile directory (profiles/v3) · keys, not pasted text

### Java

**Sections**
Purpose · Public API · Core logic · Concurrency

**Exact facts**
types · methods · constructors · fields

### SQL

**Sections**
Schema · Reads · Writes · Relationships

**Exact facts**
tables · views · indexes · INSERTs · rows

### Shell: Bash · POSIX sh · ksh93

**Sections**
Functions · Configuration · Exit codes

**Exact facts**
functions · options · traps · set -e

### Perl

**Sections**
Subs · Constants and configuration

**Exact facts**
packages · subs · modules · GetOptions

### XML: POM · Spring · XSD · XSLT

**Sections**
Definitions · Configuration · Dependencies

**Exact facts**
elements · beans · types · plugins

### Any other file: C, Python, ...

**Sections**
Fallback prompt: Purpose · Input · Output

**Exact facts**
no counters unless a profile adds them

## Own prompts for shell, Perl and XML, measured

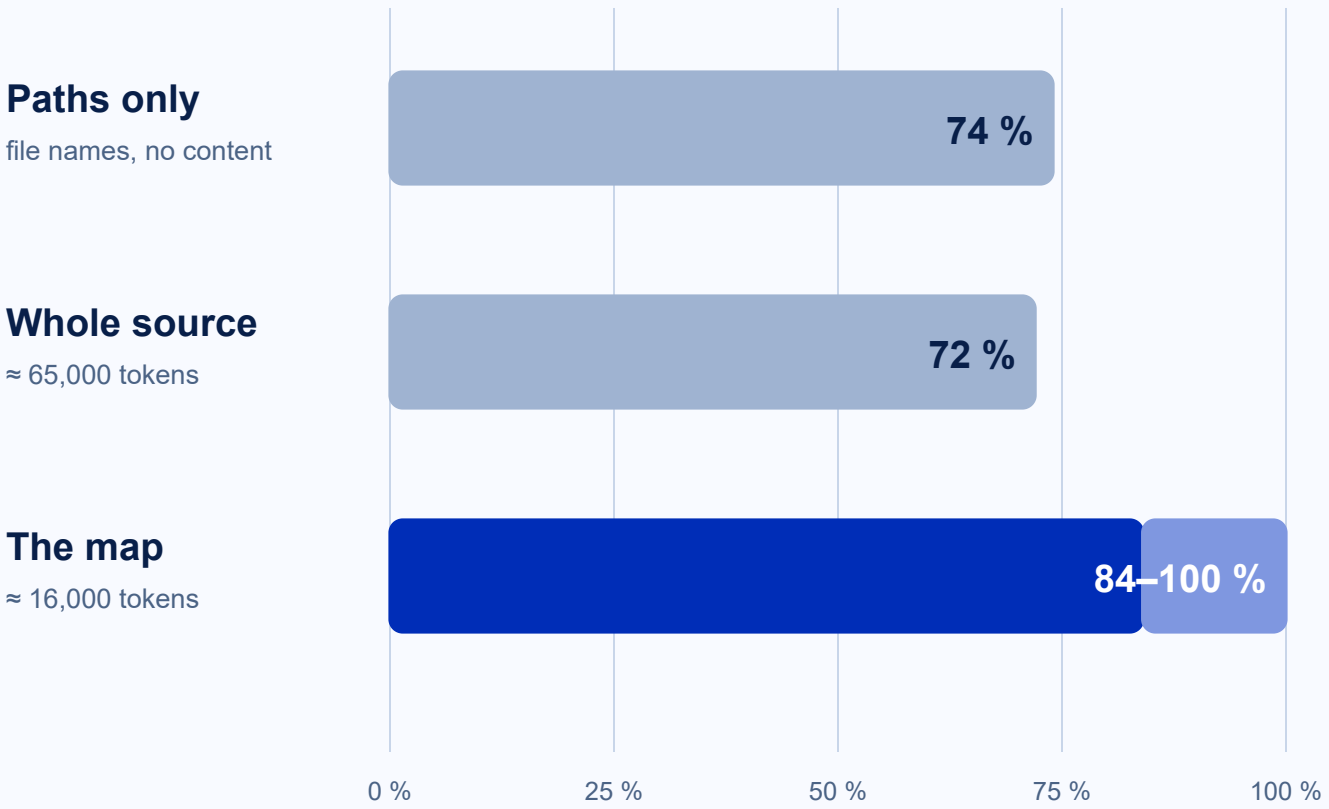
Language corpus (Perl, Bash, ksh93, POSIX sh, POM, Spring, XSD, XSLT), generic → own prompt:

**gpt-oss-20b 95.7 → 97.7 % · GPT-6 Luna 99.6 → 99.9 % (name recall + format, 3 / 6 runs)**

Fact counters ignore POD, heredocs and comments ([ignore] regions). gpt-oss-20b: 99.1 % over the six own-prompt runs without one empty provider answer.

# Does the map lead to the right file?

43 developer questions · 24 files · a separate reader model names the file that answers




The map: 32 of 35 model runs; the other three had empty or malformed summaries.

## About 4× less context

and more questions answered first than with the whole source or the paths alone.

## Keyword search does not gain

Keyword ranking (BM25, as in classic search engines): 67–91 % first over the summaries (81 % on average), 84 % over the source; 9 of 35 runs beat it.

## Missing entries hurt, not names

An empty summary costs hits; a missing name rarely does. The map is for models, not grep.

Reader: Mistral Small 3.2 24B, not one of the tested models. Questions written from the sources before any summary was read. 24 files make an easy test.

# Measured: a compact entry point for agents

October 2026 model card · 24 source files · GPT-6 Luna (hosted)

## CORPUS AND COMPLETE MARKDOWN INDEX

### Source corpus

Java · SQL · C · OpenCL · Python · Bash

**234.6 KB**

### Complete index

24 file + 12 package + 1 project Markdown

**110.2 KB** **−53% by bytes**

### Project entry: 3.06 KB · 1.30% of source

Start here, then load only relevant package / file maps.

## FILE SUMMARY BODIES

**4.09×**

Java source / summary bytes

**4.29×**

SQL source / summary bytes

**100%**

Java name recall (proxy)

One corpus · hosted via OpenRouter

Small files can produce larger summaries.

KB = 1,000 bytes; full index includes headers. Body ratios exclude headers; byte reduction is not measured token savings.

# Large files: every chunk read, facts over the whole file

1.6 MB SQL dump · 870 KB XML catalog · 283 KB Java enum · names planted in the middle of each

MAP-REDUCE OVER A FILE NO WINDOW HOLDS

**ledger_dump.sql · 1.6 MB · 30,000 rows**




map: one part summary per chunk, every chunk read

**reduce: one summary of the parts**

**Facts line: counted over the whole file**

Coverage line: what was read · summary

Over 1 MB a rule may sample chunks; the coverage line says so.

A part answered with reasoning only is read again in two halves.

## GPT-6 Luna · hosted

99.0 % mean of 3 runs (100 / 96.9 / 100 %)

Every name planted in a file's middle, in every run

## gpt-oss-20b · hosted

93.0 % mean of 3 runs (88.3 / 93.8 / 96.9 %)

Every file written; the dump's middle table or index

in every run, the enum's middle in none

## Granite 4.0 H-Tiny · local, 8 GB GPU

64–71 % (71.0 / 64.2 / 64.5 %) · every file written

No planted middle name; invents names

beside the real ones

Re-measurement 2026-11-02, three runs each; quality = name recall + format. Small local models are not yet good enough for files this large.

# Choosing a model: hosted or local

Same 24 files and scoring per row · quality proxy = expected-name recall + format

HOSTED · BEST QUALITY

## GPT-6 Luna

**100 %**

3 of 3 runs, profile v3

**6.8–7.5 s per file**

0.016–0.025 USD per run

HOSTED · BEST PER DOLLAR

## gpt-oss-20b

**99.9 %**

3 runs, profile v3

**5.3–5.6 s per file**

0.002–0.003 USD per run

LOCAL · BEST QUALITY

## gpt-oss-20b (MXFP4)

**95.8 %**

one run, v1 prompts

**33.1 s per file**

RTX 3070 Laptop 8 GB, layers auto-fitted

LOCAL · FAST

## Gemma 4 E4B, thinking off

**93 %**

4 runs: 92.8–93.2 %, v1 prompts

**11.1 s per file**

RTX 3070 Laptop 8 GB · no API fee

**Source that may not leave the machine: local.**

Approved remote processing: hosted, a few cents per run or less.

Hosted: re-measurement 2026-11-02, profile v3, OpenRouter account delta. Local: model card October 2026, v1 prompts; v3 has not run locally yet.

# Running it: one file, plan first, incremental

Maven plugin or CLI · local or hosted per model · the same engine

### One configuration file

srcmorph.yaml for the Maven plugin and the CLI:
rules, models, profile directory, output;
${name} from Maven or the system properties

### Plan and check before a model loads

Routing per file, estimated tokens, window fit;
check reads every profile file and model entry;
a broken rule or pattern fails before the run

### Incremental in CI

A changed source rewrites its entry and the
packages above it; keep the index between runs;
unchanged entries cost no model call

### A record of every run

JSON per file: rule, model, outcome, calls, tokens,
time, and the prompt and facts it was written with;
a failed file never ends the run

### Providers per model

llama.cpp in-process through java-llama.cpp (CPU and GPU backends) · OpenRouter · GitHub Copilot API Gateway
· any OpenAI-compatible /v1 server; the API key only from an environment variable

Plan and check load no model; a remote model is an explicit per-model opt-in.

# The foundation: java-llama.cpp, native inference to agent context

Maven delivery · in-process Java APIs · local and HTTP interfaces

### Applications & agents

CLI · Web · IDE via ACP
IntelliJ · VS Code (ACP extension)
Claude · Codex · local applications

### srcmorph

Java API · Maven plugin · CLI
Source → hierarchical Markdown

### HTTP & tool interfaces

OpenAI-compatible /v1 endpoints
Native server: experimental MCP

### Audio & integrations

Qwen3-TTS → WAV
Kotlin Flow · LangChain4j

## java-llama.cpp

Java model API · system prompts · model parameters · JNI / FFM

Added: JVM lifecycle + Maven classifiers + native build / patch / validation pipeline

### Native execution

Pinned llama.cpp / ggml + maintained patches + platform toolchains → DLL / SO / DYLIB

### 8 CPU library classifiers & variants

x86-64 · ARM64 · s390x | AVX2 / AVX-512 / AMX

### 15 additive GPU backend modules

CUDA · Vulkan · ROCm · SYCL · Metal · OpenCL

Merged upstream: llama.cpp #28775 (s390x), #23116 (reasoning budget) · MCP is native-server scope; ACP does not expose editor MCP servers.

# Outlook: more intelligence on the developer machine

Integrated acceleration + shared memory → practical local inference

LAPTOP HARDWARE

**Processor + shared system memory**

**CPU**
Integrated GPU
NPU / accelerator

**RAM**
Model + context

Larger shared memory pools
may make larger models practical

**Likely: acceleration inside the processor**

Also possible: external accelerators

Backend and hardware support still matter.

**LOCAL MODEL · GPT-OSS-20B**

Reasoning + tool calling on available hardware

java-llama.cpp + srcmorph + local agent

### Process large data locally

Index source and summarize large files on the workstation

### Run sensitive agent tasks locally

Use local files and tools without sending the data remotely

### Add a local review layer

A local model can review proposed remote-agent tool calls
before execution, alongside explicit access controls

Future direction; new accelerators need compatible backends. Local model review is advisory, not a security guarantee.

# Next: measure what the map is worth

Ordered by what would change a decision

## 1 Correctness of statements

Recall counts names; a summary can still state a wrong thing. Hand-check about five files per top model, or a judge model checked by hand.

## 2 A harder search test

A whole module (~200 files) with many similar files: does a better summary also search better?

## 3 Large files on local models

gpt-oss-20b and Gemma 4 on the GPU, map-reduce against sampling.

## 4 The current prompts on more models

Profile v3 has run on gpt-oss-20b and GPT-6 Luna only; the local models still use v1.

From 'What to measure next' in ai-index-benchmark/MODEL-CARD.md.

# Top 5: what the map brings

A new chat with one specific question · the map instead of a slow search through the repository

1

## A new chat starts with the whole tree in view

An empty agent knows only what it is given. project.ai.md lists every package in one line; the agent zooms into the package and file the question needs, the source only where needed.

**3.06 KB**

project entry · 1.3 % of the source

2

## Questions in plain words find the right file first

43 developer questions: the reader names the answering file first in 84–100 % with the map, 74 % with paths, 72 % with the whole source. Keyword search (BM25) does not gain from the map.

**84–100 %**

right file first (hit@1)

3

## About a quarter of the input tokens

≈ 16,000 tokens of map instead of ≈ 65,000 tokens of source for the same question, with more right answers: input cost falls with the tokens, the window stays free for the task.

**≈ 4×**

less context per question

4

## Large and unfamiliar files become answerable

A 1.6 MB SQL dump or an 870 KB XML catalog fits no window: srcmorph reads every chunk and counts exact facts over the whole file. GPT-6 Luna named every planted middle name, every run.

**99.0 %**

large files · mean of 3 runs

5

## Cheap to build, current in CI, local if needed

The 24-file index costs 0.002–0.003 USD hosted (gpt-oss-20b, 99.9 %); a changed file rewrites only its entry and the packages above it. Source that may not leave the machine: local, no API fee.

**0.003 USD**

per index run · 24 files

## Map first, source where needed

A specific question in a new chat finds the right file more often first, with about a quarter of the context, instead of opening the repository file by file.

24-file benchmark corpus (234.6 KB), 43 questions, reader Mistral Small 3.2 24B · a large project is not yet measured; a ~200-file module is the next search test.

# Native binaries for more environments

Appendix · platform-specific Maven delivery for CPU, GPU and Java applications

### CPU architectures

x86-64 · ARM64 · s390x

14 x86 / 8 Linux ARM CPU variants

### GPU backends

CUDA · ROCm · Vulkan

SYCL · OpenCL · Metal

### Target platforms

Linux · Windows · macOS · Android

Native DLL / SO / DYLIB libraries

## 23 native Maven classifiers

8 library classifiers + 15 additive GPU modules · OS / architecture / backend

ggml selects a compatible CPU variant at runtime

### Platform fixes & validation

JNI / C ABI · Android · s390x QEMU

Native libraries + packaged consumer tests

### Contributions to llama.cpp

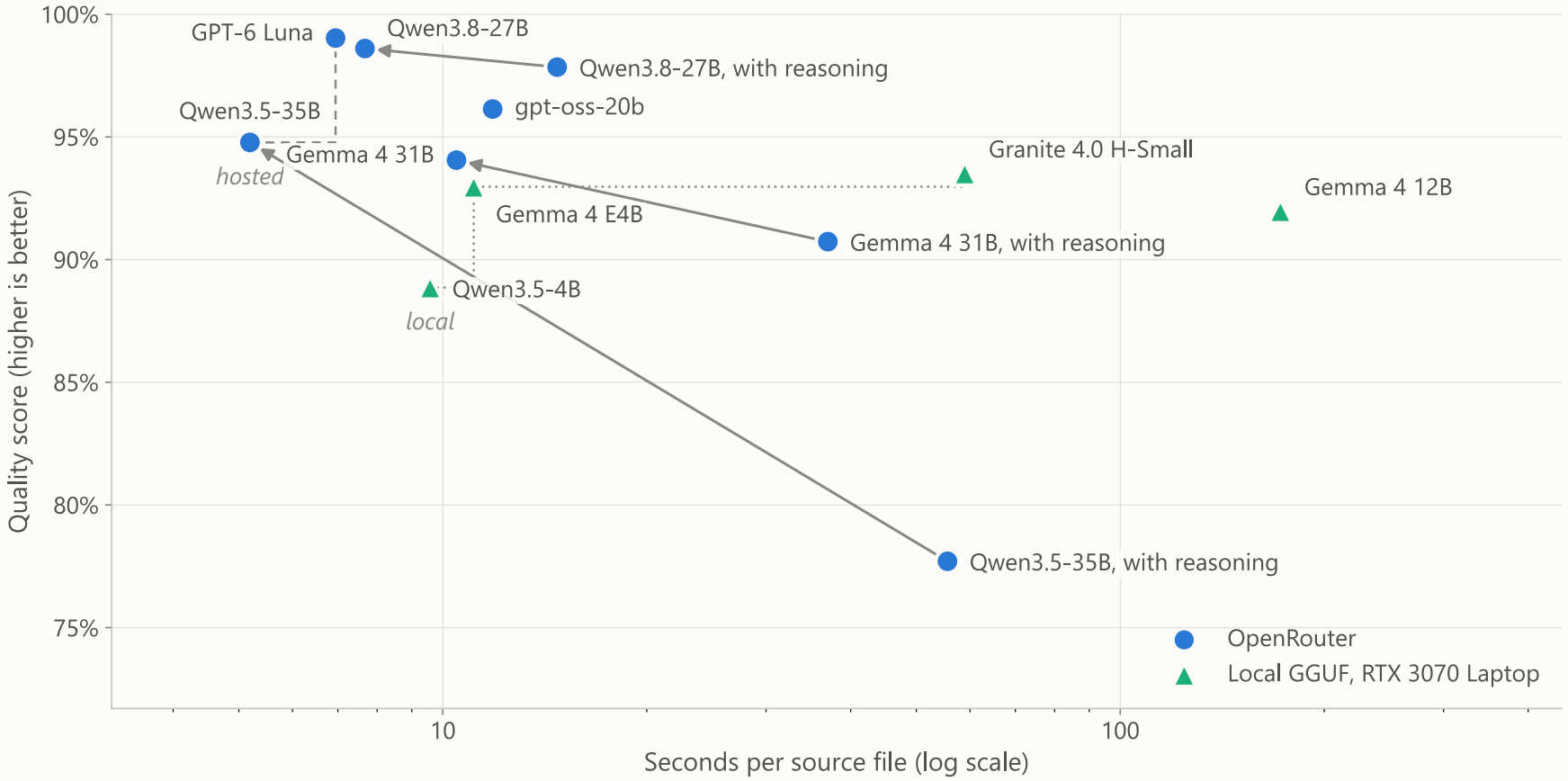
Merged: #28775 s390x · #23116 reasoning budget

CPU variant measurements identify tuning opportunities

Native CPU variants are inside classifier packages. Platform support and runtime requirements vary.

# Quality versus indexing speed

Appendix · 12 of the model card's 35 runs · hosted models and local GGUF on a laptop GPU




## How to read it

Up: stronger quality proxy
Left: fewer seconds per file
Arrow: reasoning switched off

## Reasoning off

Qwen3.5-35B: 78% → 95%
55.6 s → 5.2 s per file

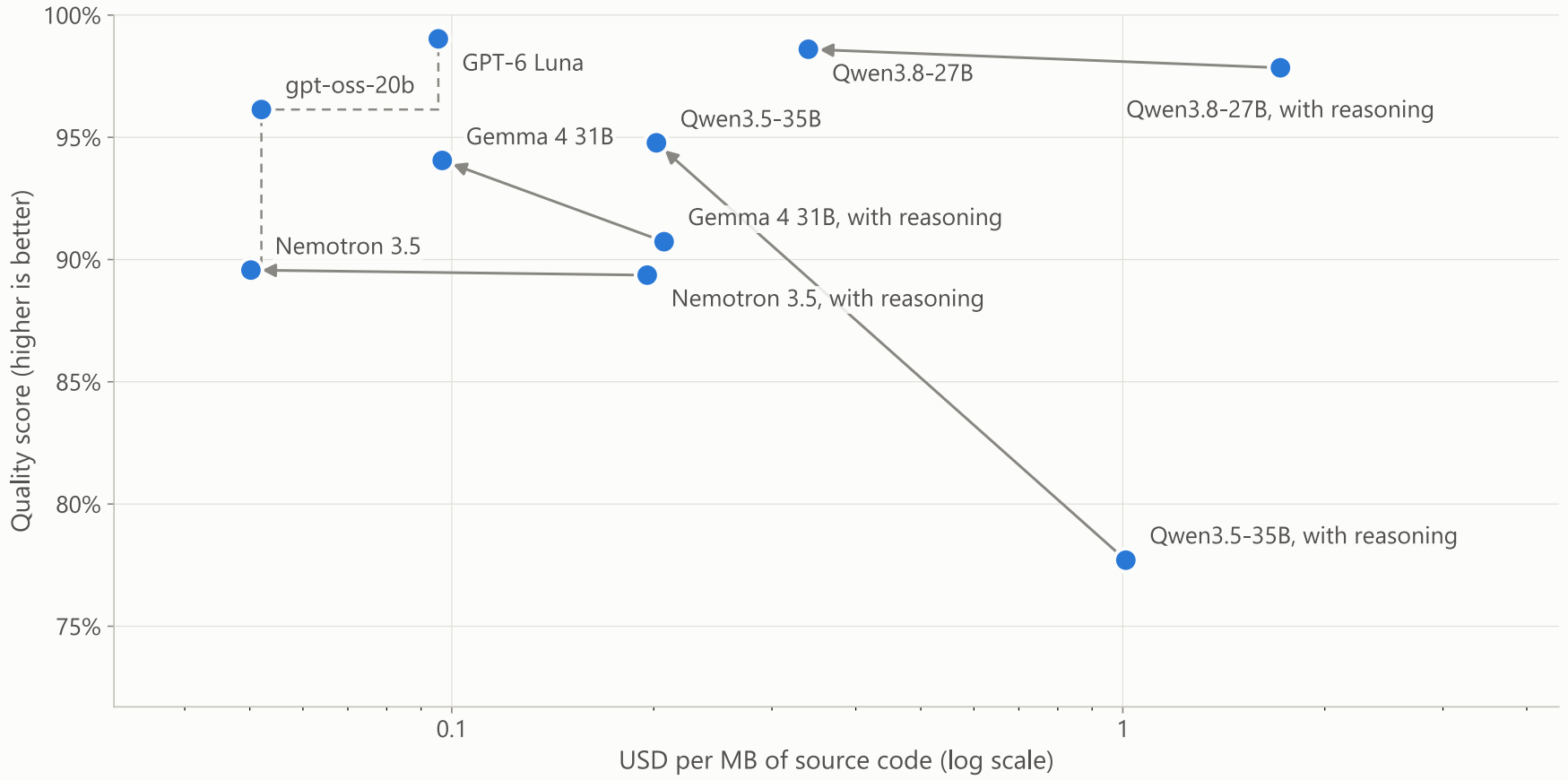
## Gemma 4 E4B · local

93% quality proxy
11.1 s per file, laptop GPU

Model-card data · Hosted and local weights differ · All 35 runs: ai-index-benchmark/MODEL-CARD.md

# Quality versus hosted cost

Appendix · 10 OpenRouter runs · indexing cost per MB of source




### How to read it

Up: stronger quality proxy
Left: lower hosted cost
Arrow: reasoning switched off

### Best per dollar: GPT-OSS-20B

97% quality proxy, 3 runs
About $0.05 per MB

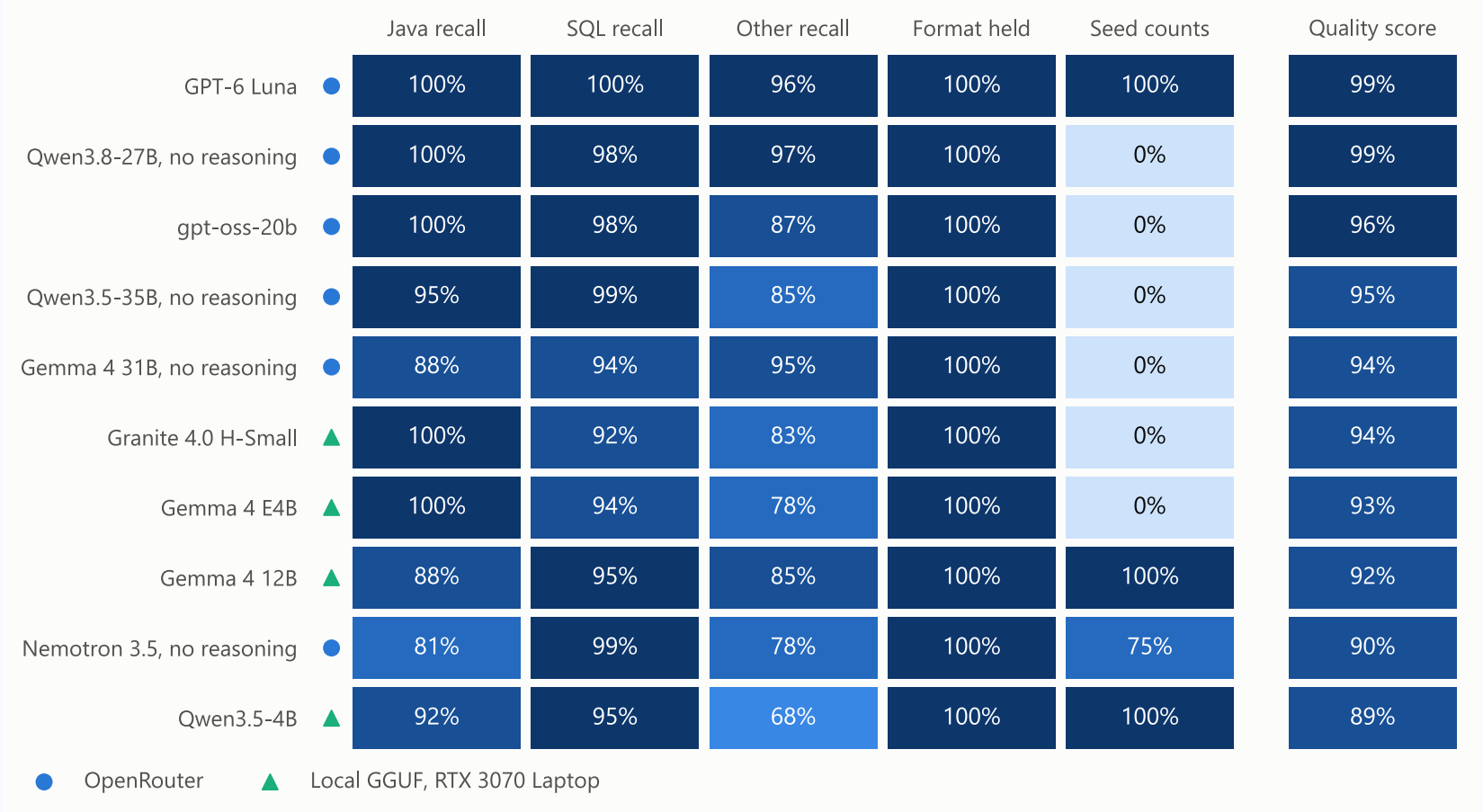
### Reasoning off

2–5× cheaper
at equal or better quality

Model-card data · Prices and workloads vary · Local runs have no API fee, but hardware and energy cost

# Model quality profile

Appendix · the ten models of these charts, best setting each · Java · SQL · other · format · seed counts




## How to read it

Score = mean of the first four columns.
Seed counts: does the summary of the seed-data SQL file state its four row counts (15 / 110 / 50 / 150)? Shown, not scored.

## GPT-6 Luna · remote

100% Java / SQL name recall
All four seed counts

## Exact counts are rare

3 of 10 models state all four row counts. That is why srcmorph writes an exact facts line beside every summary.

Model-card data · Name recall is a proxy, not factual verification